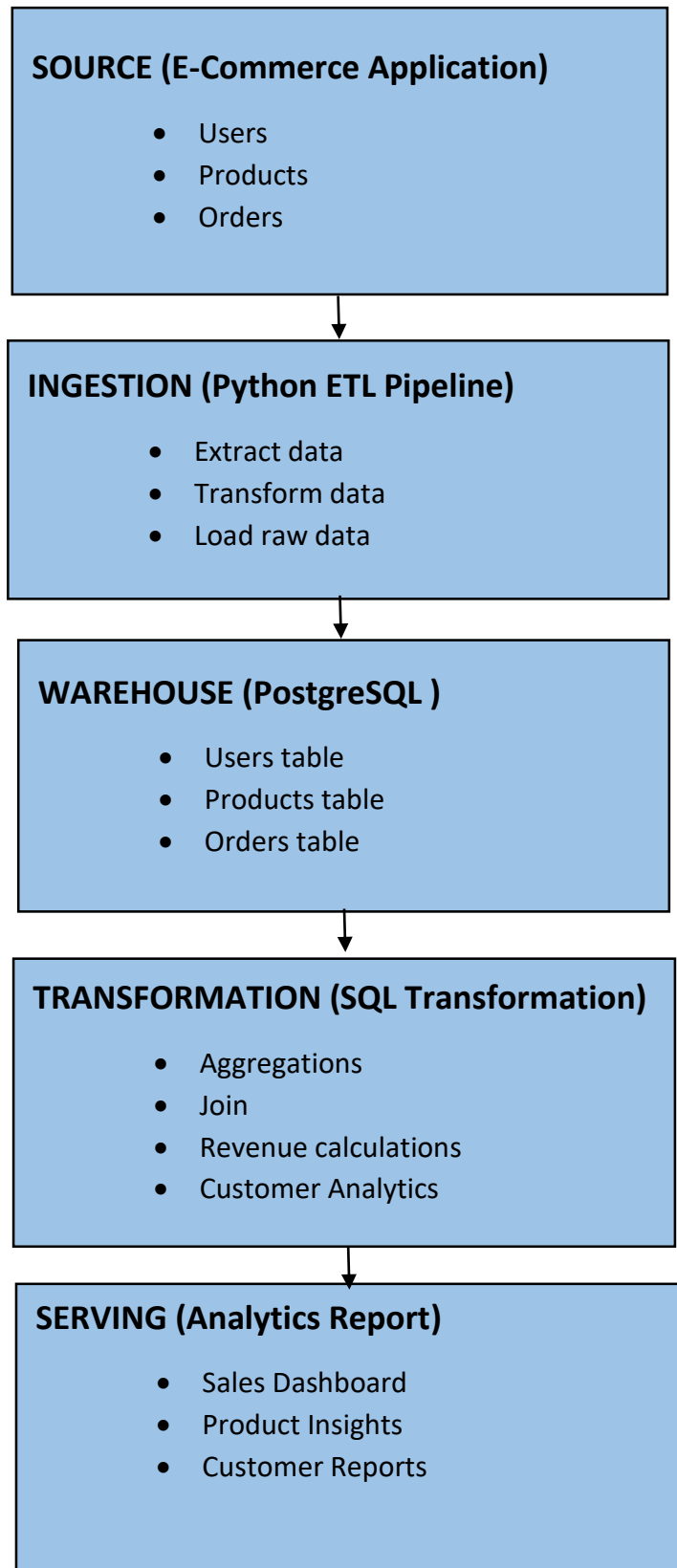


Draw a modern data stack



DESCRIPTION OF DATA STACK

1.1 Sources

Sources are systems where data is generated.

Examples:

- Application databases
- APIs
- Logs
- Sensors
- CSV/JSON files

Example:

An e-commerce website generates customer and order data.

1.2 Ingestion

Ingestion moves data from sources to storage systems.

Types

Batch Ingestion

Data moves at intervals (hourly/daily).

Example:

Daily sales reports.

Streaming Ingestion

Data moves continuously in real time.

Example:

Live traffic monitoring.

Common tools:

- Apache Kafka
- Airflow
- Python scripts

1.3 Data Warehouse

A data warehouse stores cleaned and structured data for analytics.

Features:

- Fast SQL queries
- Historical analysis
- Reporting and dashboards

Examples:

- Snowflake
- BigQuery
- Redshift

1.4 Transformation

Transformation converts raw data into usable business data.

Tasks:

- Cleaning duplicates
- Joining tables
- Aggregations
- Standardization

Example:

Combining customer and order tables to calculate revenue.

1.5 Serving Layer

The serving layer provides data to users and applications.

Consumers:

- Dashboards
- Reports
- APIs
- Machine learning models

Roles and Ownership

Different teams manage different parts of the data platform.

Role	Main Responsibility	Tasks	Skills / Tools	Ownership
Data Engineer	Builds and maintains data pipelines and infrastructure	Build pipelines, move data, manage warehouse, ensure reliability, orchestration	SQL, Python, Spark, Airflow, Cloud	Pipelines, ingestion, warehouse loading, reliability
Analytics Engineer	Turns warehouse data into business-ready models	Clean data, write SQL, define metrics, data modeling	dbt, SQL, Snowflake, BigQuery	Metrics, reporting models, semantic layer
Platform Engineer	Maintains infrastructure systems	Manage cloud, CI/CD, Kubernetes, automation	Kubernetes, Docker, CI/CD tools, Cloud	Infrastructure, deployment, networking
ML Engineer	Builds and deploys ML systems	Deploy models, build features, monitor, train models	Python, TensorFlow, PyTorch, ML tools	ML pipelines, model serving, feature store
Data Analyst	Generates business insights	Dashboards, reporting, analysis, trends	SQL, Power BI, Tableau, Excel	Reports, dashboards, KPIs

Batch vs Streaming

Type	Description	Advantages	Disadvantages	Example
Batch Processing	Processes data at scheduled intervals	Simple, Cheap, Easy to maintain	Not real-time	Daily payroll processing
Streaming Processing	Processes data continuously in real time	Instant updates, Real-time monitoring	Expensive, Complex infrastructure	Fraud detection systems

Warehouse vs Lake vs Lakehouse

Feature	Data Warehouse	Data Lake	Data Lakehouse
Data Type	Structured	Structured + Unstructured	Structured + Unstructured
Cost	High	Low	Medium
Best Use	BI and reporting	Raw data storage	Hybrid analytics
Example	Snowflake, BigQuery	Amazon S3, Hadoop	Databricks, Delta lake

OLTP vs OLAP

Feature	OLTP	OLAP
Full Form	Online Transaction Processing	Online Analytical Processing
Purpose	Operational systems	Analytical systems
Examples	Banking, E-commerce checkout	Sales analysis, BI dashboards
Query Type	Fast inserts/updates	Large queries, aggregations
Data Focus	Current data	Historical data

Row vs Columnar Storage

Feature	Row Storage	Columnar Storage
Storage Style	Stores full rows together	Stores data column by column
Best For	Transactional systems	Analytical systems
Performance	Fast row retrieval	Fast aggregation queries
Compression	Low	High (better compression)

Example of row storage:

ID	NAME	AGE
1	Ram	20

Example of columnar storage:

IDs :1,2,3

NAMES: Ram, Sita, Hari

AGEs: 20, 21, 23

Why analytics is its own database problem?

Analytics is its own database problem because it has very different needs than normal application databases. OLTP systems are designed for fast, small transactions like inserting or updating single records (e.g., banking or online shopping). OLAP systems are designed for analytics, where large amounts of data are scanned to find patterns, trends, and summaries (e.g., total sales, reports). Because analytics requires reading huge datasets, aggregations, and historical data, it needs different storage, indexing, and processing methods (like columnar storage and data warehouses). If we try to use the same system for both transactions and analytics, performance becomes slow.